Python Engineering

WEEK 04 - OOP, Files & Testing | CLASS 13 - Classes & Objects | EX21

Objetivo: introducir programacion orientada a objetos construyendo una clase que represente el estado de un movimiento vertical. Aprenderemos class, object, __init__, self, atributos y metodos.

Ruta

```
01_python_bootcamp/
`-- week_04_oop_files_testing/
    `-- class_13_classes_objects/
        |-- notes/
        `-- exercises/
            `-- ex21_vertical_state_class/
                |-- vertical_state.py
                `-- main.py
```

Valores de referencia

Caso	Valores iniciales	Referencia
Earth object	h=100 m, v=20 m/s, t=0 s, g=9.81	specific energy = 1181.0 J/kg
Moon object	h=100 m, v=20 m/s, t=0 s, g=1.62	specific energy = 362.0 J/kg
Earth after dt=0.1 s	-	h=102.0, v=19.019, t=0.1
Moon after dt=0.1 s	-	h=102.0, v=19.838, t=0.1

BLOCK A - Crear la clase

```
class VerticalState:
    def __init__(self, h, v, t=0.0, g=9.81):
        self.h = h
        self.v = v
        self.t = t
        self.g = g
```

La clase define un nuevo tipo de objeto. __init__ configura el estado inicial de cada objeto creado.

BLOCK B - Metodo specific_energy()

```
def specific_energy(self):
    return self.g * self.h + 0.5 * self.v**2
```

BLOCK C - Metodo state_vector()

```
def state_vector(self):
    return np.array([self.h, self.v, self.t])
```

BLOCK D - Metodo is_ascending()

```
def is_ascending(self):
    return self.v > 0.0
```

BLOCK E - Metodo euler_step()

```
def euler_step(self, dt):
    self.h = self.h + self.v * dt
    self.v = self.v - self.g * dt
    self.t = self.t + dt
```

Este metodo modifica el propio objeto. Antes de llamarlo, el objeto tiene un estado; despues, sus atributos h, v y t contienen el nuevo estado.

BLOCK F - Crear dos objetos independientes

```
earth_state = VerticalState(
```

```
    h=100.0,
    v=20.0
)

moon_state = VerticalState(
    h=100.0,
    v=20.0,
    g=1.62
)
```

Ambos objetos pertenecen a la misma clase, pero guardan atributos independientes.

BLOCK G - Usar los metodos

```
earth_energy = earth_state.specific_energy()
moon_energy = moon_state.specific_energy()

earth_before = earth_state.state_vector()
moon_before = moon_state.state_vector()

earth_state.euler_step(0.1)
moon_state.euler_step(0.1)

earth_after = earth_state.state_vector()
moon_after = moon_state.state_vector()
```

BLOCK H - main.py

```
from vertical_state import VerticalState

def main():
    # Create Earth and Moon objects
    # Call methods
    # Compare their independent states
    # Print results

if __name__ == '__main__':
    main()
```

VALIDACION FINAL

```
Explain in comments:
- What is a class?
- What is an object or instance?
- What does __init__ do?
- What does self refer to?
- What is the difference between a parameter h and self.h?
- What is an attribute?
- What is a method?
- Why can earth_state and moon_state have different g values?
- What changes inside the object when euler_step() is called?
- Why is earth_state.specific_energy() different from VerticalState.specific_energy?
- What advantages does this representation have over passing h, v, t and g to every function?
```

Criterio de cierre

EX21 estara completado cuando puedas crear varios objetos de una misma clase, entender que cada uno mantiene su propio estado y distinguir con claridad clase, objeto, atributo y metodo.